

PILHA E FILA

ENCADEADAS

Resolução de Problemas
Estruturados em Computação



Antes de entendermos como implementar as estruturas de **Pilha** e **Fila**, precisamos entender o que é o conceito de encadeamento em listas lineares.

Quando estudamos sobre memória, entendemos que nossos objetos são instanciados no espaço de memória chamado **memória heap**.

Nós vimos que esse espaço de memória não possui organização no armazenamento dos dados, ou seja, os dados ficam todos “espalhados” nesse espaço de memória.

Em **estruturas encadeadas** nós instanciamos cada um dos elementos **individualmente**. Esses elementos são armazenados na **memória heap**, e para conectar esses elementos, utilizamos a técnica de encadeamento.

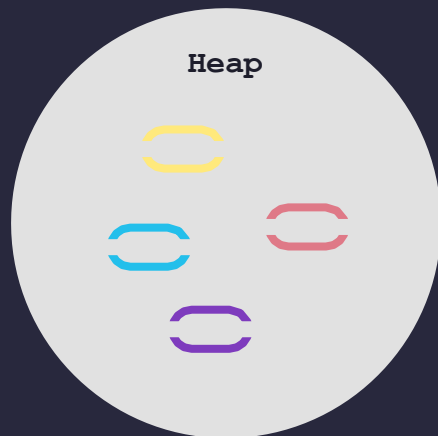
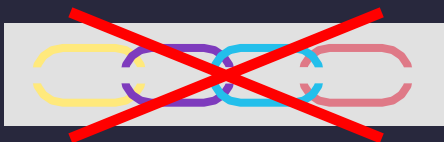
/Encadeamento

Imagine que cada elemento de uma lista linear é o elo de uma corrente:



Quando os elos de uma corrente não estão conectados, eles ficam soltos e espalhados. A mesma coisa acontece com uma lista linear. Se não conectarmos um elemento ao outro, não sabemos qual é o primeiro elemento, e nem a sequência em que os elementos foram adicionados.

/Encadeamento



Os elementos de uma lista, nunca vão ficar lado a lado como em um array estático (lembram que estamos falando de memória), mas conseguimos simular esse comportamento fazendo com que um elemento aponte na direção do próximo. Vamos ver como isso é organizado em termos de algoritmo

/Encadeamento

Cada elo das nossas listas, ou “caixinha na memória”, é representado por uma estrutura chamada **nó** (node). Essa estrutura contém duas informações: o dado que está sendo inserido, e uma referência para o elemento próximo ou anterior (ou ambos).

Node

```
E element;  
Node next/previous;
```

```
Node (E e);
```

/Adicionando Elementos

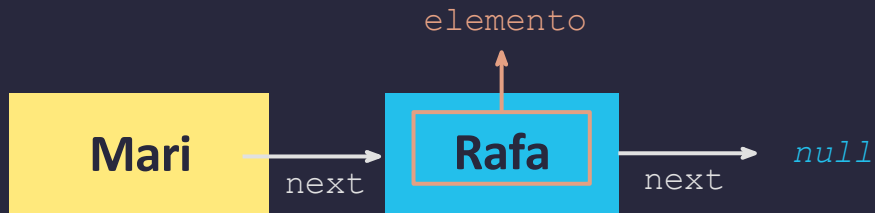
Vamos usar como exemplo a sequência de strings: "Mari", "Rafa" e "Josi":



A primeira string que adicionamos é "Mari". Portanto, na nossa estrutura **Node**, o elemento será "Mari" e o next será nulo. O next é nulo, pois não adicionamos nenhuma outra string à lista, portando esse **Node** não possui nenhum elemento posterior a ele.

/Adicionando Elementos

Agora vamos adicionar a string "Rafa":



Na segunda string que adicionamos, o elemento é "Rafa" e o next é nulo, pois, da mesma que forma que no primeiro **Node**, ele não possui um elemento posterior. Mas, o **Node** do elemento "Mari" agora tem seu next apontando para o **Node** do elemento "Rafa", pois a string "Rafa" foi adicionada posteriormente à string "Mari".

/Adicionando Elementos

Para fixar bem o conceito, vamos adicionar a última string ("Josi"):



A mesma situação acontece nesse exemplo e segue acontecendo enquanto strings forem adicionadas à lista. Nesta representação os elementos parecem estar lado a lado, mas na verdade eles estão espalhados na **memória heap** conforme vimos anteriormente.

/Referência ao anterior

A ordem do encadeamento também pode ser feita de forma reversa:



A lógica do encadeamento "reverso" é a mesma, porém a referência que o **Node** vai armazenar, é uma referência para o objeto anterior. Nós também conseguimos fazer o encadeamento duplo, mas vamos estudar isso mais pra frente.

Pilha

/Pilha - Lista Encadeada

A pilha é um tipo de lista encadeada que tem como princípio a sigla LIFO - Last In First Out (Último que entra é o primeiro que sai). Você pode imaginar essa estrutura como uma pilha de roupas, pratos, livros, etc. A lógica é a mesma: os elementos são empilhados na ordem em que são adicionados e, ao remover, o elemento do topo da pilha é o que será removido.



/Pilha - Lista Encadeada

Os principais métodos de uma pilha são:

push(E e)

Método que empilha um elemento qualquer em uma estrutura do tipo pilha. Segue a lógica de adição em uma lista encadeada que vimos nos slides anteriores.

pop()

Método que desempilha um elemento qualquer de uma estrutura do tipo pilha. Esse método sempre vai remover e retornar o elemento no topo da pilha (último).

/Pilha - Removendo Elementos

Para remover o **último elemento** de uma pilha encadeada:



Na pilha, sempre teremos uma referência para o objeto no topo. Essa referência sempre vai apontar para o último **Node** da pilha. Como o método de remoção retorna o elemento do **Node**, precisamos guardar esse valor string em uma variável.

/Pilha - Removendo Elementos

Para remover o **último elemento** de uma pilha encadeada:



Em seguida, precisamos mover o **top** para o **Node** anterior, e podemos definir o **Node** do elemento "Josi" como *null* (o que não é necessário pois o **top** já indica o final da pilha, mas não tem problema fazer também). Por fim, retornamos o elemento do **Node** removido.

Fila

/Fila - Lista Encadeada

A fila é um tipo de lista encadeada que tem como princípio a sigla FIFO - First In First Out (Primeiro que entra é o primeiro que sai). Você pode imaginar essa estrutura como uma fila de pessoas, pedidos, processos, etc. A lógica é a mesma: os elementos são enfileirados na ordem em que são adicionados e, ao remover, o elemento do início da fila é o que será removido.



/Fila - Lista Encadeada

Os principais métodos de uma fila são:

enqueue(E e)

Método que enfileira um elemento qualquer em uma estrutura do tipo fila. Segue a lógica de adição em uma lista encadeada que vimos nos slides anteriores.

dequeue()

Método que desenfileira um elemento qualquer de uma estrutura do tipo fila. Esse método sempre vai remover e retornar o primeiro elemento da fila.

/Fila - Removendo Elementos

Para remover o **último elemento** de uma fila encadeada:

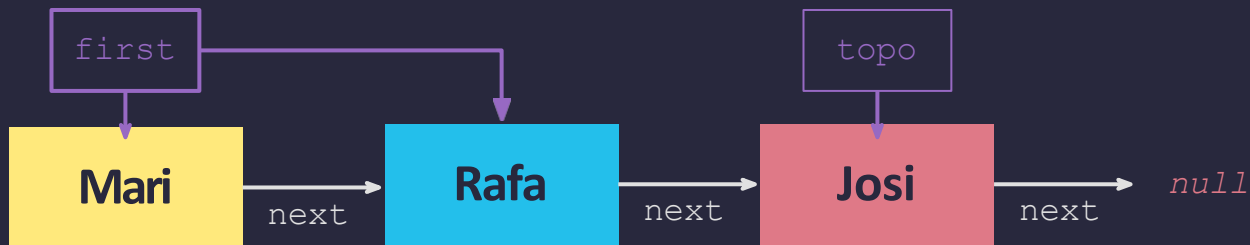


Na fila, além da referência no topo, temos uma referência no início. Essa referência sempre vai apontar para o primeiro **Node** da fila.

Como o método de remoção retorna o elemento do **Node**, precisamos guardar o valor string do primeiro **Node** em uma variável.

/Fila - Removendo Elementos

Para remover o **último elemento** de uma fila encadeada:



O processo de remoção da fila é semelhante à remoção da pilha, porém, a remoção é feita pelo primeiro elemento. Então, movemos o `first` para o próximo **Node**.

/Entendimento Básico

FILAS (Queue)

Estrutura de dados FIFO (First In, First Out) → o primeiro a entrar é o primeiro a sair.

Exemplo: fila de banco, fila no supermercado, impressão de documentos.

Fila Simples

Implementada como vetor (array).

Inserção (enqueue) no final e remoção (dequeue) no início.

/Entendimento Básico

FILAS (Queue)

Fila Simples

Exemplo: atendimento de senha em banco → senha 1 atendida, mas a posição do array fica vazia.

/Exemplo em Código

```
1  class FilaSimples {
2      private int[] dados;
3      private int inicio, fim, tamanho;
4
5      public FilaSimples(int capacidade) {
6          dados = new int[capacidade];
7          inicio = fim = tamanho = 0;
8      }
9
10     public void enqueue(int valor) {
11         if (fim == dados.length) throw new RuntimeException("Fila cheia!");
12         dados[fim++] = valor;
13         tamanho++;
14     }
15
16     public int dequeue() {
17         if (tamanho == 0) throw new RuntimeException("Fila vazia!");
18         int valor = dados[inicio++];
19         tamanho--;
20         return valor;
21     }
22 }
```

/Entendimento Básico

FILAS (Queue)

Fila Circular

Resolve o problema da fila simples.

Quando chega ao final do vetor, o fim volta para o início se houver espaço.

Mais eficiente no uso da memória.

Exemplo: rodízio de processos na CPU (escalonamento circular).

/Exemplo em Código

```
1  class FilaCircular {
2      private int[] dados;
3      private int inicio, fim, tamanho;
4
5      public FilaCircular(int capacidade) {
6          dados = new int[capacidade];
7          inicio = fim = tamanho = 0;
8      }
9
10     public void enqueue(int valor) {
11         if (tamanho == dados.length) throw new RuntimeException("Fila cheia!");
12         dados[fim] = valor;
13         fim = (fim + 1) % dados.length;
14         tamanho++;
15     }
16
17     public int dequeue() {
18         if (tamanho == 0) throw new RuntimeException("Fila vazia!");
19         int valor = dados[inicio];
20         inicio = (inicio + 1) % dados.length;
21         tamanho--;
22         return valor;
23     }
24 }
```

/Entendimento Básico

FILAS (Queue)

Fila Encadeada

Usa nós ligados por ponteiros.

Não tem tamanho fixo → cresce dinamicamente.

Mais flexível que vetor.

Exemplo: fila de impressora onde os documentos podem variar muito de tamanho.

/Exemplo em Código

```
1 class No {
2     int valor;
3     No prox;
4     No(int v) { valor = v; }
5 }
6
7 class FilaEncadeada {
8     private No inicio, fim;
9
10    public void enqueue(int valor) {
11        No novo = new No(valor);
12        if (fim != null) fim.prox = novo;
13        fim = novo;
14        if (inicio == null) inicio = novo;
15    }
16
17    public int dequeue() {
18        if (inicio == null) throw new RuntimeException("Fila vazia!");
19        int valor = inicio.valor;
20        inicio = inicio.prox;
21        if (inicio == null) fim = null;
22        return valor;
23    }
24 }
```

/Entendimento Básico

FILAS (Queue)

Fila Duplamente Encadeada (Deque)

Cada nó aponta para o próximo e o anterior.

Permite inserção/remoção em ambas as extremidades.

É chamada de Deque (Double Ended Queue).

Exemplo: navegadores (voltar e avançar páginas).

Ouro exemplo: agendamento de tarefas em sistemas operacionais.

/Exemplo em Código

```
1  import java.util.Deque;
2  import java.util.LinkedList;
3
4  ∨ class ExemploDeque {
5  ∨      public static void main(String[] args) {
6          Deque<Integer> deque = new LinkedList<>();
7          deque.addFirst(1); // início
8          deque.addLast(2);  // fim
9          System.out.println(deque.removeFirst()); // remove do início
10         System.out.println(deque.removeLast());  // remove do fim
11     }
12 }
```

/Entendimento Básico

PILHAS (Stack)

Estrutura LIFO (Last In, First Out) → o último a entrar é o primeiro a sair.

Exemplo: pilha de pratos, desfazer ações (Ctrl+Z), execução de chamadas de funções.

Pilha Simples (com vetor)

Implementada em array.

Push = insere no topo.

Pop = remove do topo.

/Exemplo em Código

```
1  class PilhaSimples {  
2      private int[] dados;  
3      private int topo;  
4  
5      public PilhaSimples(int capacidade) {  
6          dados = new int[capacidade];  
7          topo = -1;  
8      }  
9  
10     public void push(int valor) {  
11         if (topo == dados.length - 1) throw new RuntimeException("Pilha cheia!");  
12         dados[++topo] = valor;  
13     }  
14  
15     public int pop() {  
16         if (topo == -1) throw new RuntimeException("Pilha vazia!");  
17         return dados[topo--];  
18     }  
19 }
```

/Entendimento Básico

PILHAS (Stack)

Pilha Encadeada

Implementada com nós ligados.

Mais flexível, sem limite fixo.

Exemplo: chamadas recursivas em execução de programas.

/Exemplo em Código

```
1  class NoPilha {
2      int valor;
3      NoPilha prox;
4      NoPilha(int v) { valor = v; }
5  }
6
7  class PilhaEncadeada {
8      private NoPilha topo;
9
10     public void push(int valor) {
11         NoPilha novo = new NoPilha(valor);
12         novo.prox = topo;
13         topo = novo;
14     }
15
16     public int pop() {
17         if (topo == null) throw new RuntimeException("Pilha vazia!");
18         int valor = topo.valor;
19         topo = topo.prox;
20         return valor;
21     }
22 }
```

/Entendimento Básico

PILHAS (Stack)

Pilha Duplamente Encadeada

Mais rara.

Permite navegar tanto para o anterior quanto para o próximo.

Usada em cenários onde se precisa percorrer a pilha nos dois sentidos.

Exemplo: Navegação entre abas ou histórico de programas.

Em Java, pode ser feita com Deque também.

/Entendimento Básico

QUANDO USAR FILA x PILHA

Usar Fila quando:

Ordem de atendimento precisa ser **FIFO**.

Exemplos: impressão de documentos, atendimento em sistemas, escalonamento de processos.

Usar Pilha quando:

Ordem de atendimento precisa ser **LIFO**.

Exemplos: histórico de ações, execução de funções, sistemas de desfazer/refazer.

/Entendimento Básico

QUEM DECIDE QUAL USAR?

O problema/algoritmo é quem decide, não o programador "por gosto".

O programador analisa a lógica de acesso necessária:

- Se precisa atender por ordem de chegada → Fila.
- Se precisa resgatar o último colocado → Pilha.

Depois, escolhe a implementação mais eficiente (simples, circular, encadeada, etc.), considerando:

- Memória (vetor fixo ou lista dinâmica).
- Eficiência (fila circular evita desperdício).
- Complexidade do problema.

/Relacionando com Tuplas

QUAL É A RELAÇÃO COM TUPLAS?

Tuplas não são filas nem pilhas.

Tuplas podem ser os elementos armazenados dentro de filas ou pilhas, quando precisamos guardar mais de uma informação ao mesmo tempo.

Elas ajudam a representar dados compostos de forma simples.

Muitas vezes, cada elemento da fila/pilha não é só um valor, mas sim um conjunto de informações. É aí que entram tuplas (ou estruturas equivalentes).

/Relacionando com Tuplas

QUAL É A RELAÇÃO COM TUPLAS?

Exemplo prático em fila: fila de atendimento no hospital: cada posição guarda uma tupla (nome, prioridade, horário_chegada). Assim, ao enfileirar, não guardo só o nome, mas um pacote de dados.

Exemplo prático em pilha: histórico de navegação: cada posição da pilha pode ser uma tupla (url, data_acesso, posição_scroll). Ao dar "voltar", desempilho a tupla completa, não apenas a URL.

/Relacionando com Tuplas

QUAL É A RELAÇÃO COM TUPLAS?

Em Java, não existe tupla nativa, mas usamos classes como `Pair`, `Record` ou `AbstractMap.SimpleEntry`.

Uma tupla serve para agrupar dados heterogêneos sem a necessidade de criar uma classe.

Ex.: `(id, nome, idade)`.

/Exercícios

Fila Simples (Fila de Resgate)

Na Liga dos Heróis, pessoas em perigo chamam ajuda. Cada pedido de resgate recebe um número e entra numa fila de atendimento. O herói disponível pega sempre o primeiro pedido da fila.

Implemente uma fila simples que receba pedidos de resgate (inteiros) e permita:

- Inserir novo pedido na fila.
- Atender o próximo pedido.
- Mostrar pedidos na fila.

/Exercícios

Fila Circular (Bat-Sinais de Gotham)

O Comissário Gordon dispara o Bat-Sinal quando precisa do Batman. Porém, o projetor só aguenta guardar 3 pedidos simultâneos. Se todos os espaços estiverem cheios, um pedido novo só entra se o Batman atender um sinal antigo. Para reaproveitar espaço, o sistema usa fila circular.

Implemente uma fila circular para armazenar até 3 chamados de socorro.

/Exercícios

Fila Encadeada (Sala da Justiça)

Na Sala da Justiça, vários heróis chegam para falar com o Superman. Como não dá para prever quantos vão aparecer, a lista precisa ser dinâmica. Quem chega vai para o final da fila e o Superman chama sempre o primeiro.

Implemente uma fila encadeada para registrar heróis que querem falar com o Superman.

/Exercícios

Deque (Arena dos Vingadores)

Na arena de treino dos Vingadores, os heróis podem entrar tanto pela esquerda quanto pela direita do campo. Da mesma forma, também podem sair por qualquer lado. Esse comportamento é exatamente o de uma fila duplamente encadeada (Deque).

Implemente um Deque para controlar entrada e saída dos heróis na arena.

/Exercícios

Pilha Simples (Capitão América e o Escudo)

O Capitão América armazena ataques bloqueados com seu escudo em uma pilha. Cada novo ataque é colocado no topo. Quando ele contra-ataca, remove o último ataque bloqueado.

Implemente uma pilha simples para simular os bloqueios de ataques.

/Exercícios

Pilha Encadeada (Homem-Aranha no Prédio)

O Homem-Aranha sobe um prédio lançando teias. Cada andar que ele sobe é adicionado na pilha. Quando ele decide descer, vai removendo os andares do topo até chegar ao chão.

Implemente uma pilha encadeada para simular os movimentos do Homem-Aranha no prédio.

/Exercícios

Pilha com Deque (Doutor Estranho e os Portais)

O Doutor Estranho abre portais em sequência, cada novo portal em cima do anterior. Quando decide fechar, o último aberto é o primeiro a desaparecer.

Implemente uma pilha usando Deque para simular abertura e fechamento de portais.